

ANÁLISIS DEL RETO

Juan Sebastián Rojas Marín, 202610140, js.rojasm123@uniandes.edu.co

Katy Teran Mercado, 202614247, k.teranm@uniandes.edu.co

Juan José Ricon Santos, 202615789, jj.rincons1@uniandes.edu.co

Requerimiento <<2>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```
def req_2(catalog, min_price, max_price):  
    """  
    Retorna el resultado del requerimiento 2  
    """  
    # TODO: Modificar el requerimiento 2  
    start_time = get_time()  
  
    orders = catalog["orders"]  
    total = lt.size(orders)  
  
    count = 0  
    sum_discount = 0  
    sum_marketing = 0  
    sum_price = 0  
  
    recent_order = None  
    min_amount_order = None  
    max_amount_order = None  
  
    for i in range(total):  
        order = lt.get_element(orders, i)  
  
        price = float(order["Price_per_Box"])  
  
        if price < min_price or price > max_price:  
            continue  
  
        count += 1  
  
        discount = float(order["Discount_Pct"])  
        marketing = float(order["Marketing_Spend"])  
        amount = float(order["Amount"])  
        date = order["Order_Date"]  
  
        sum_discount += discount  
        sum_marketing += marketing  
        sum_price += price
```

```

if recent_order is None:
    recent_order = order

else:
    recent_date = recent_order["Order_Date"]

    if date > recent_date:
        recent_order = order

    elif date == recent_date:
        if amount > float(recent_order["Amount"]):
            recent_order = order

if min_amount_order is None:
    min_amount_order = order

else:
    current_min_amount = float(min_amount_order["Amount"])
    current_min_price = float(min_amount_order["Price_per_Box"])

    if amount < current_min_amount:
        min_amount_order = order

    elif amount == current_min_amount:
        if price < current_min_price:
            min_amount_order = order

if max_amount_order is None:
    max_amount_order = order

else:
    current_max_amount = float(max_amount_order["Amount"])
    current_max_price = float(max_amount_order["Price_per_Box"])

    if amount > current_max_amount:
        max_amount_order = order

    elif amount == current_max_amount:
        if price < current_max_price:
            max_amount_order = order

```

```

if count > 0:
    avg_discount = sum_discount / count
    avg_marketing = sum_marketing / count
    avg_price = sum_price / count
else:
    avg_discount = 0
    avg_marketing = 0
    avg_price = 0

end_time = get_time()

result = {
    "count": count,
    "avg_discount": avg_discount,
    "avg_marketing": avg_marketing,
    "avg_price": avg_price,
    "recent_order": recent_order,
    "min_amount_order": min_amount_order,
    "max_amount_order": max_amount_order
}

return result, delta_time(start_time, end_time)

```

Este requerimiento recorre los pedidos almacenados en un ArrayList y filtra aquellos cuyo Price_per_Box se encuentra entre el precio mínimo (min_price) y el precio máximo (max_price) ingresados. Para los pedidos que cumplen el filtro se calcula la cantidad total, el promedio de Discount_Pct, el promedio de Marketing_Spend y el promedio de Price_per_Box. Además, se identifica el pedido más reciente; si dos pedidos tienen la misma fecha, se selecciona el de mayor Amount. También se identifica el pedido de menor y de mayor Amount dentro del rango; si existe empate en Amount, se selecciona el que tenga menor Price_per_Box.

Entrada	Estructuras de datos del modelo, precio mínimo por caja (min_price) y precio máximo por caja (max_price).
Salidas	Tiempo de ejecución en milisegundos, cantidad de pedidos encontrados, promedio de descuento, promedio de inversión en marketing, promedio de precio por caja, pedido más reciente, pedido de menor Amount y pedido de mayor Amount.
Implementado (Sí/No)	Sí. Implementado por Katy Yulieth Teran Mercado

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener la lista de pedidos y su tamaño	$O(1)$
Recorrer todos los pedidos del ArrayList	$O(n)$
Filtrar, acumular valores y comparar pedidos en cada iteración	$O(1)$
Calcular los promedios	$O(1)$
Construir el resultado	$O(1)$
TOTAL	$O(n)$

Análisis

El comportamiento de este requerimiento depende principalmente de la cantidad de pedidos cargados, ya que para encontrar cuáles tienen un Price_per_Box dentro del rango ingresado es necesario revisar todos los elementos de la lista. En cada pedido se verifica si cumple con ese rango y, si no lo cumple, se continúa con el siguiente. Cuando sí cumple, se usan sus datos para calcular los promedios y se hacen las comparaciones necesarias para encontrar el pedido más reciente, el de menor Amount y el de mayor Amount. Estas operaciones son sumas y comparaciones directas, por lo que tienen complejidad $O(1)$ para cada pedido.

Como en el peor de los casos se deben revisar los n pedidos almacenados, la complejidad total del requerimiento es $O(n)$. Esto se puede ver en la prueba realizada con chocolate_sale_100_ptc.csv, usando un rango de precio por caja entre 20 y 100. En esa prueba se encontraron 106 pedidos que cumplían la condición y el requerimiento tomó 53.847 milisegundos. Al usar el archivo con la mayor cantidad de datos, se puede observar cómo se comporta el algoritmo cuando debe revisar todos los pedidos para encontrar los que cumplen con el rango indicado.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.